DeVinci Blockchain

FROM FIRST COMMIT TO DEPLOYMENT

GIT & GITHUB WORKSHOP



THURSDAY 22 OCT · 17:30 – 19:00 · NANTERRE

IIM – DIGITAL SCHOOL · NANTERRE

Before we start

01 **Check that Git is installed**

Open a terminal (Windows: **Git Bash**) and type `git --version`

02 **Sign in to GitHub**

No account yet? `github.com/signup` — takes 2 minutes.

03 **Open VS Code**

To edit your files. Not installed? `code.visualstudio.com`



github.com/alibenyezza/dvb-git-workshop

Tonight's agenda

17:30 — 8 MIN
Intro
Settle in, check your tools.

17:38 — 15 MIN
Concepts
Git, GitHub, commits, branches.

17:53 — 15 MIN
Lab 1
Your first repo, locally.

18:08 — 12 MIN
Break 🧉 + pro habits
Commit messages, versions.

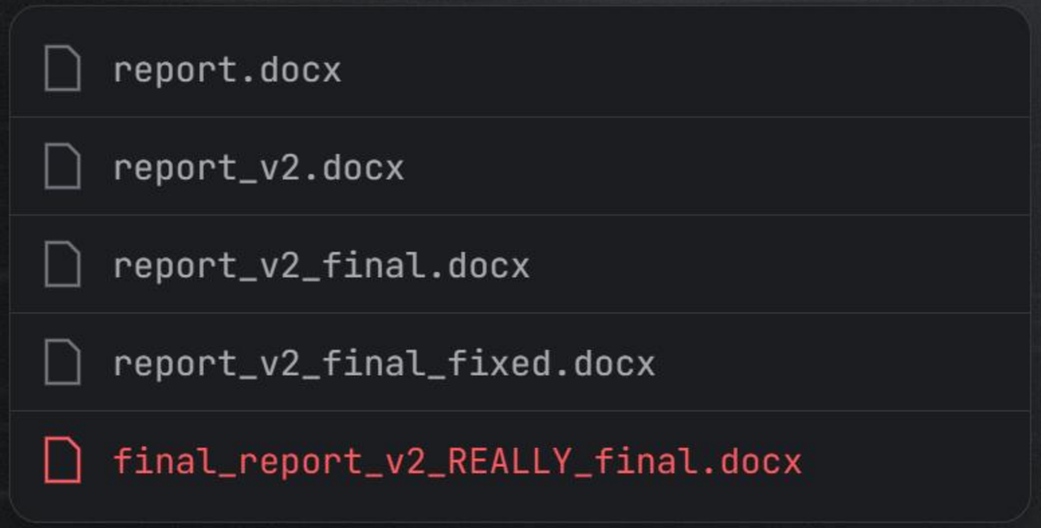
18:20 — 22 MIN
Lab 2
Your card on the DVB wall, via a Pull Request.

18:42 — 8 MIN
Deployment
How it went live: CI/CD.

18:50 — 8 MIN
Kahoot 🏆
12 questions, top 3 win.

COMMON THREAD — By the end, **your card is live** on the DVB wall — deployed by you.

Why Git?



Git keeps **every version** of your project: who changed what, when, and why.

- **History** — see how the whole project evolved.
- **Undo** — go back to a version that worked.
- **Teamwork** — everyone moves forward without overwriting each other.

Git vs GitHub

Git

LOCAL · OPEN SOURCE

- A **program** installed on your computer
- Records your project's **history**
- Works **offline**, from the command line
- Commands: `git commit` `git branch`

CREATED IN 2005 BY LINUS TORVALDS

VS

GitHub

CLOUD · PLATFORM

- A **website** that hosts your Git repos
- **Pull Requests** & code review
- **Actions**: automated tests and deployment
- **Pages**: free website hosting

ALTERNATIVES: GITLAB, BITBUCKET

GIT IS THE ENGINE — GITHUB IS THE PLATFORM BUILT AROUND IT

The 4 areas of Git

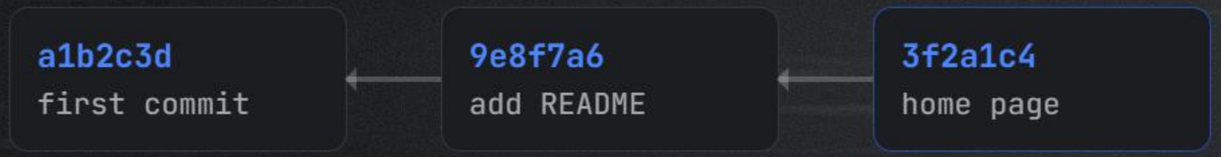


What is a commit?

```
git log

commit 3f2a1c4e8b9d0a7c (HEAD → main)
Author: Ada Lovelace <ada@mail.com>
Date:   Thu Oct 22 17:58:12 2026 +0200

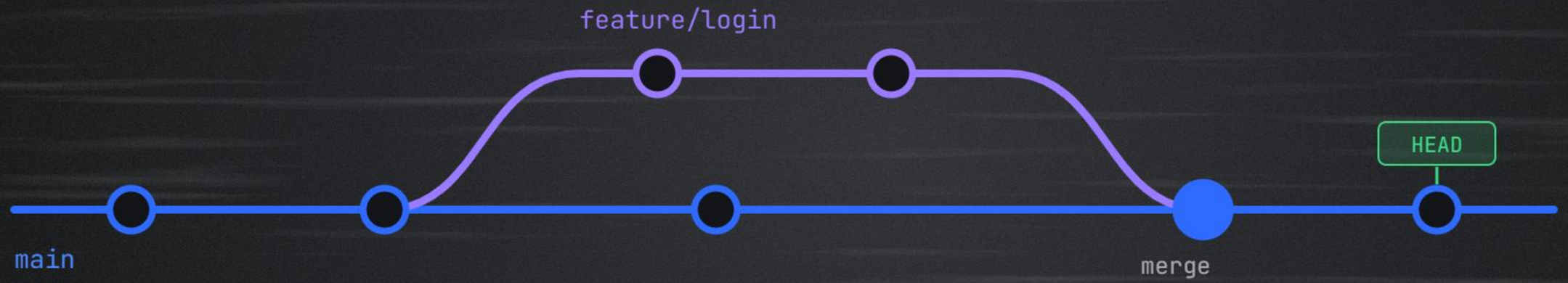
    feat: add the home page
```



A **snapshot of the whole project** at a point in time.

- **Hash** — unique ID (`3f2a1c4`)
- **Author & date** — who, when
- **Message** — why
- **Parent** — the previous commit: a chain

Branches




```
$ git switch -c feature/login   # create the branch and switch to it
$ git switch main               # go back to main
$ git merge feature/login       # bring the work into main
```

Branches in real teams



NAME YOUR BRANCHES

```
feature/user-login
# a new feature
fix/navbar-overflow
# a bug fix
hotfix/payment-crash
# urgent fix in production
release/1.4.0
# prepare a version
```

Lowercase, dashes, short and explicit.

Git Flow
main + staging + dev. Big teams, planned releases.

Trunk-based
main + short branches. Most startups.

Lab 1 — Your first repo

15 MIN · LOCAL

01 **Set up your identity**
Your name and email, just once.

02 **Create a repo**
A folder + `git init`

03 **Make your first commit**
A `README.md`, add, commit.

04 **Create a branch**
Work on `feature/bio`

05 **Merge into main**
Bring your work back to the main branch.

★ **Bonus: a conflict**
Change the same line on 2 branches.

STUCK? RAISE YOUR HAND, I'LL COME TO YOU

Lab 1 — the commands

1 · setup & first commit

```
$ cd ~/Desktop
$ git config --global user.name "First Last"
$ git config --global user.email "you@mail.com"
$ mkdir my-first-repo
$ cd my-first-repo
$ git init -b main
$ code .
▸ VS Code: new file README.md
  write "# Hello DVB" → Ctrl+S
$ git status        # red: not staged
$ git add README.md
$ git status        # green: staged
$ git commit -m "first commit"
$ git log --oneline
```

2 · branch & merge

```
$ git switch -c feature/bio
▸ VS Code: add a line "I do dev."
  in README.md → Ctrl+S
$ git add README.md
$ git commit -m "feat: add my bio"
$ git switch main   # the line is gone!
$ git merge feature/bio # it's back
$ git log --oneline --graph --all

# bonus: undo a change not added yet
$ git restore README.md
```

Merge conflicts — don't panic

```
$ git merge feature/bio
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and
then commit the result.
```

README.md

```
# Hello DVB
<<<<<<< HEAD
I do design.
=======
I do dev.
>>>>>>> feature/bio
```

01 Open the file. VS Code offers **Accept Current / Incoming / Both**.

02 Keep the right version, **delete the markers** `<<<` `===` `>>>`

03 `git add README.md` then `git commit`

↺ Undo everything: `git merge --abort`

Break 

Back at **18:15**

07:00

USE THE BREAK TO

- **Finish Lab 1**: raise your hand if you're stuck
- **Log in** to github.com
- Get ready for **Lab 2**: your card on the wall

Writing good commits

`type: what the commit does, in one sentence`

feat new feature	**fix** bug fix	**docs** documentation	**refactor** restructures without changing behaviour
style formatting	**test** adds or fixes tests	**chore** maintenance, config	**ci** CI pipeline

```
✗ update
✗ fix stuff
✗ aaaaaa
```

```
✓ feat: add ada to the wall
✓ fix: correct the link to the site
✓ docs: explain plan B in the README
```

Versions & tags

1.4.2

MAJOR MINOR PATCH

- MAJOR — breaks compatibility
- MINOR — new feature
- PATCH — bug fix

tags

```
$ git tag -a v1.0.0 -m "First release"
$ git push origin v1.0.0
$ git tag
v1.0.0

# on GitHub: Releases → Draft a new release
```

Collaborating on GitHub

GITHUB

1

Fork

Your copy of the repo, on your account.

YOUR LAPTOP

2

Clone

`git clone`

YOUR LAPTOP

3

Branch

`git switch -c`

YOUR LAPTOP

4

Commit

`git add`

`git commit`

YOUR LAPTOP

5

Push

`git push`

GITHUB

6

Pull Request

You propose your work.

GITHUB

7

Merge

After review and a green check.

You can't write to my repo: you work on **your fork**, then **propose** your changes with a Pull Request.

Lab 2 — the Builders Wall

22 MIN

Add **your card** to the DVB wall: one file `builders/your-username.json`, one Pull Request, and it goes live.

builders/ada.json

```
{
  "github": "ada",
  "name": "Ada L.",
  "school": "IIM",
  "role": "Dev",
  "emoji": "🚀",
  "message": "My first commit!"
}
```

github.com/alibenyezza/dvb-git-workshop

Lab 2 — step by step

Git Bash · Terminal

```
① github.com/alibenyezza/dvb-git-workshop
  → "Fork" (top right) → "Create fork"
$ cd ~/Desktop
$ git clone https://github.com/your-username/dvb-git-workshop.git
$ cd dvb-git-workshop
$ git switch -c add/your-username
$ cp builders/_template.json builders/your-username.json
$ code builders/your-username.json
② VS Code: fill in your card (right) → Ctrl+S
$ git add builders/your-username.json
$ git commit -m "feat: add your-username to the wall"
$ git push -u origin add/your-username
③ GitHub: "Compare & pull request"
  → "Create pull request"
```

builders/your-username.json

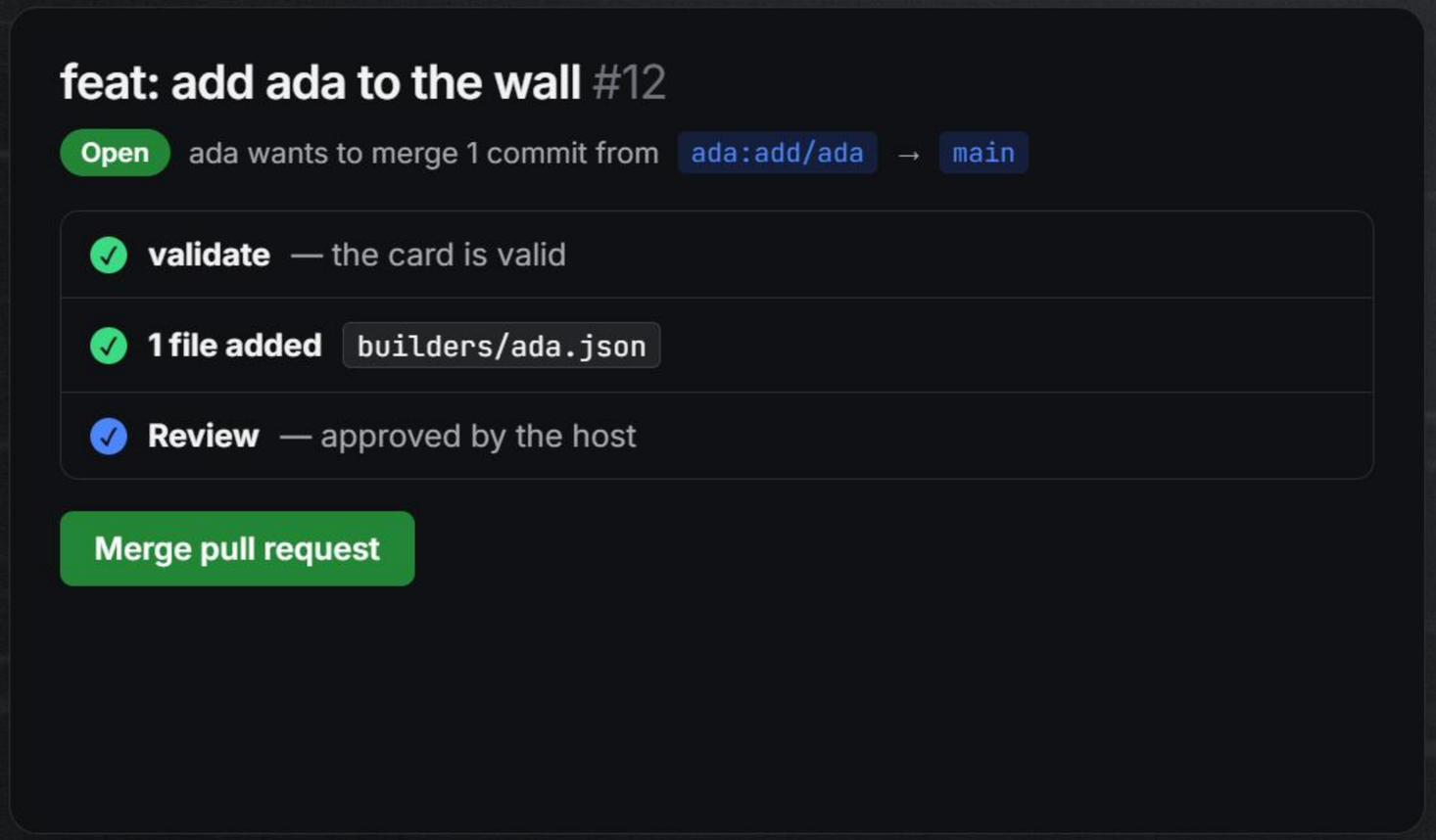
```
{
  "github": "your-username",
  "name": "First L.",
  "school": "IIM",
  "role": "Dev",
  "emoji": "🚀",
  "message": "My first commit!"
}
```

role: Dev · Design · Data
Product · Business · Curious
1 emoji · message ≤ 80 chars

PLAN B · NO TERMINAL

On the repo: **Add file** → **Create new file** → `builders/your-username.json` → paste the card → **Propose changes**

The Pull Request



WHAT THE REVIEWER CHECKS

- One single file, in the **right place**
- The **validate** check is green
- A clear **commit message**

Red check? Read the error, fix it, `commit` + `push`: the PR updates itself.

The wall fills up

● LIVE

alibenyezza.github.io/dvb-git-workshop

AL — **Ada L.** 🚀
IIM · DEV
My first commit!

LT — **Linus T.** 🐧
ESILV · DATA
git push, and off it goes.

GH — **Grace H.** ✨
EMLV · PRODUCT
My first merged PR.

MH — **Margaret H.** 🌙
IIM · DESIGN
Hello DVB!

AT — **Alan T.** 🧠
ESILV · CURIOUS
Branches finally make sense.

+ your card here

MOCK-UP · THE REAL WALL IS LIVE DURING THE EVENT

How does it get deployed?

1. **Merge into main**
   Your PR is accepted.
2. **GitHub Actions starts**
   A robot runs the workflow `deploy`
3. **Check & build**
   Validates every card and assembles them into a site.
4. **GitHub Pages publishes**
   The site is updated in ~1 minute.

.github/workflows/deploy.yml (simplified)

```yaml
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: node scripts/validate.mjs
      - run: node scripts/build.mjs
      - uses: actions/upload-pages-artifact@v3
  deploy:
    needs: build
    steps:
      - uses: actions/deploy-pages@v4
```

Protecting main

Pull Request required

REQUIRED

No change reaches main without a PR.

Green check

REQUIRED

The `validate` check must pass before merging.

Review

RECOMMENDED

At least one person reviews before merging.

No force push

REQUIRED

Never rewrite main's history.

IF YOU PUSH STRAIGHT TO MAIN

```
$ git push origin main
remote: error: GH013: Repository rule
violations found for refs/heads/main.
 ! [remote rejected] main → main (push
declined due to repository rule violations)
```

Settings: **Settings → Rules → Rulesets**

Done already? Bonus challenges

1

Review a neighbour

Leave a constructive comment on their PR.

2

Update your card

New branch, new PR: change your message.

3

Undo a commit

`git revert <hash>`

4

Explore the history

`git log --graph` · `git blame`

5

Try rebasing

On your own repo: `git rebase main`

6

Your portfolio online

Repo `your-username.github.io` + Pages

Key takeaways

01

Commit often

Small commits with clear messages.

02

One branch per task

main always stays clean.

03

Everything goes through a PR

Reviewed, discussed, then merged.

04

Checks are your friends

A robot checks every push.

05

Push = backup

Your work is online and shared.

What's next?

FREE FOR STUDENTS

GitHub Student Developer Pack

Pro tools for free, with your student email.

education.github.com/pack

GO FURTHER

Learn & practise

The official book, the docs, and a game to master branches.

git-scm.com/book

docs.github.com

learngitbranching.js.org

DEVINCI BLOCKCHAIN

Upcoming events

Workshops, talks, hackathons: follow DVB so you don't miss a thing.

DeVinci Blockchain

Kahoot!

Grab your phone → **kahoot.it**

12 QUESTIONS ABOUT TONIGHT · 20 S EACH

🏆 TOP 3 WIN

GAME PIN

Enter the PIN, pick a nickname, and you're in.

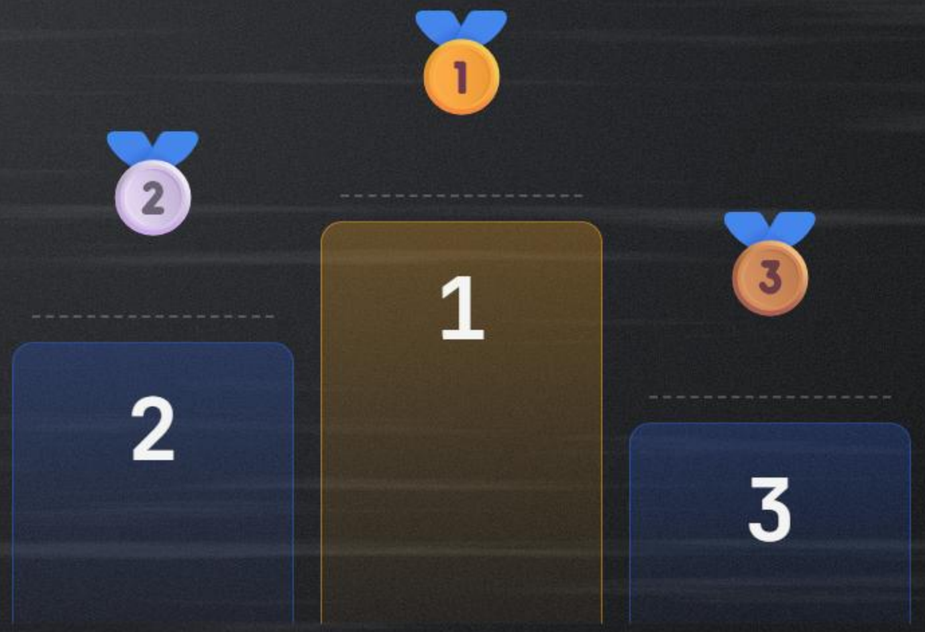
DeVinci Blockchain

Thank you!

Congrats to the **Kahoot winners** 🏆

Your feedback matters: **Luma will send you a short form** after the event.

📷 GROUP PHOTO IN FRONT OF THE WALL



FROM FIRST COMMIT TO DEPLOYMENT